Algoritmos de Ordenação: testes de mesa

PSC II · Engenharia de Software · Código em C++ (arquivos 01 a 05 do repositório)

Entrada usada nos cinco testes: `[5, 2, 9, 1, 7, 3]` (n = 6). Usei a mesma entrada para todos para dar para comparar os números no final.

Como contei

- Comparações: toda vez que dois valores do array são comparados entre si (o contador sobe antes do if).
- Trocas no Bubble, Selection e Quick: cada troca real entre dois elementos. Se o elemento já está no lugar (ou ia ser trocado com ele mesmo), o código não troca e não conta.
- Trocas no Insertion: cada deslocamento `arr[j+1] = arr[j]`. O total dá o mesmo número de trocas entre vizinhos que o Bubble faria.
- Trocas no Merge: cada escrita no array durante a intercalação, já que esse algoritmo não troca elementos de verdade.
- Quick Sort com pivô no último elemento, para o teste não depender de sorteio.
- Nas tabelas, as colunas Comp. e Trocas são acumuladas. No Merge e no Quick, as chamadas com 0 ou 1 elemento só retornam e por isso não aparecem como linha.

1. Bubble Sort

Passada	j	Compara	Resultado	Ação	Array depois	Comp.	Trocas
1	0	arr[0]=5 > arr[1]=2	sim	troca arr[0] e arr[1]	[2, 5, 9, 1, 7, 3]	1	1
1	1	arr[1]=5 > arr[2]=9	não	-	[2, 5, 9, 1, 7, 3]	2	1
1	2	arr[2]=9 > arr[3]=1	sim	troca arr[2] e arr[3]	[2, 5, 1, 9, 7, 3]	3	2
1	3	arr[3]=9 > arr[4]=7	sim	troca arr[3] e arr[4]	[2, 5, 1, 7, 9, 3]	4	3
1	4	arr[4]=9 > arr[5]=3	sim	troca arr[4] e arr[5]	[2, 5, 1, 7, 3, 9]	5	4
fim da passada 1: trocou = true							
2	0	arr[0]=2 > arr[1]=5	não	-	[2, 5, 1, 7, 3, 9]	6	4
2	1	arr[1]=5 > arr[2]=1	sim	troca arr[1] e arr[2]	[2, 1, 5, 7, 3, 9]	7	5
2	2	arr[2]=5 > arr[3]=7	não	-	[2, 1, 5, 7, 3, 9]	8	5
2	3	arr[3]=7 > arr[4]=3	sim	troca arr[3] e arr[4]	[2, 1, 5, 3, 7, 9]	9	6
fim da passada 2: trocou = true							
3	0	arr[0]=2 > arr[1]=1	sim	troca arr[0] e arr[1]	[1, 2, 5, 3, 7, 9]	10	7
3	1	arr[1]=2 > arr[2]=5	não	-	[1, 2, 5, 3, 7, 9]	11	7
3	2	arr[2]=5 > arr[3]=3	sim	troca arr[2] e arr[3]	[1, 2, 3, 5, 7, 9]	12	8
fim da passada 3: trocou = true							
4	0	arr[0]=1 > arr[1]=2	não	-	[1, 2, 3, 5, 7, 9]	13	8
4	1	arr[1]=2 > arr[2]=3	não	-	[1, 2, 3, 5, 7, 9]	14	8
fim da passada 4: trocou = false -> nenhuma troca, sai do laço (break)							

Saída: `[1, 2, 3, 5, 7, 9]` **Comparações:** 14 **Trocas:** 8

Na passada 4 não houve nenhuma troca, então `trocou` ficou false e o laço parou. A passada 5 nem chegou a rodar.

2. Selection Sort

Passada	j	Compara	Resultado	minIdx depois	Comp.	Trocas
passada 1: i = 0, minIdx = 0 (arr[0] = 5)						
1	1	arr[1]=2 < arr[minIdx]=5	sim	1	1	0
1	2	arr[2]=9 < arr[minIdx]=2	não	1	2	0
1	3	arr[3]=1 < arr[minIdx]=2	sim	3	3	0
1	4	arr[4]=7 < arr[minIdx]=1	não	3	4	0
1	5	arr[5]=3 < arr[minIdx]=1	não	3	5	0
fim da passada 1: menor = 1 (estava na posição 3); troca arr[0] e arr[3] -> `[1, 2, 9, 5, 7, 3]` (trocas = 1)						
passada 2: i = 1, minIdx = 1 (arr[1] = 2)						
2	2	arr[2]=9 < arr[minIdx]=2	não	1	6	1
2	3	arr[3]=5 < arr[minIdx]=2	não	1	7	1
2	4	arr[4]=7 < arr[minIdx]=2	não	1	8	1
2	5	arr[5]=3 < arr[minIdx]=2	não	1	9	1
fim da passada 2: menor = 2 já está na posição 1; sem troca -> `[1, 2, 9, 5, 7, 3]` (trocas = 1)						
passada 3: i = 2, minIdx = 2 (arr[2] = 9)						
3	3	arr[3]=5 < arr[minIdx]=9	sim	3	10	1
3	4	arr[4]=7 < arr[minIdx]=5	não	3	11	1
3	5	arr[5]=3 < arr[minIdx]=5	sim	5	12	1
fim da passada 3: menor = 3 (estava na posição 5); troca arr[2] e arr[5] -> `[1, 2, 3, 5, 7, 9]` (trocas = 2)						
passada 4: i = 3, minIdx = 3 (arr[3] = 5)						
4	4	arr[4]=7 < arr[minIdx]=5	não	3	13	2
4	5	arr[5]=9 < arr[minIdx]=5	não	3	14	2
fim da passada 4: menor = 5 já está na posição 3; sem troca -> `[1, 2, 3, 5, 7, 9]` (trocas = 2)						
passada 5: i = 4, minIdx = 4 (arr[4] = 7)						
5	5	arr[5]=9 < arr[minIdx]=7	não	4	15	2
fim da passada 5: menor = 7 já está na posição 4; sem troca -> `[1, 2, 3, 5, 7, 9]` (trocas = 2)						

Saída: `[1, 2, 3, 5, 7, 9]` **Comparações:** 15 **Trocas:** 2

Mesmo com o array já ordenado depois da passada 3, o algoritmo continua fazendo todas as comparações: são sempre n(n-1)/2 = 15. As únicas trocas foram nas passadas 1 e 3.

3. Insertion Sort

i	j	Compara	Resultado	Ação	Array depois	Comp.	Trocas
i = 1: chave = arr[1] = 2, j = 0							
1	0	arr[0]=5 > chave=2	sim	arr[1] = arr[0] (desloca)	[5, 5, 9, 1, 7, 3]	1	1
sai do laço (j < 0); arr[0] = 2 -> [2, 5, 9, 1, 7, 3]							
i = 2: chave = arr[2] = 9, j = 1							
2	1	arr[1]=5 > chave=9	não	para, achou o lugar	[2, 5, 9, 1, 7, 3]	2	1
sai do laço (arr[1] <= chave); arr[2] = 9 -> [2, 5, 9, 1, 7, 3]							
i = 3: chave = arr[3] = 1, j = 2							
3	2	arr[2]=9 > chave=1	sim	arr[3] = arr[2] (desloca)	[2, 5, 9, 9, 7, 3]	3	2
3	1	arr[1]=5 > chave=1	sim	arr[2] = arr[1] (desloca)	[2, 5, 5, 9, 7, 3]	4	3
3	0	arr[0]=2 > chave=1	sim	arr[1] = arr[0] (desloca)	[2, 2, 5, 9, 7, 3]	5	4
sai do laço (j < 0); arr[0] = 1 -> [1, 2, 5, 9, 7, 3]							
i = 4: chave = arr[4] = 7, j = 3							
4	3	arr[3]=9 > chave=7	sim	arr[4] = arr[3] (desloca)	[1, 2, 5, 9, 9, 3]	6	5
4	2	arr[2]=5 > chave=7	não	para, achou o lugar	[1, 2, 5, 9, 9, 3]	7	5
sai do laço (arr[2] <= chave); arr[3] = 7 -> [1, 2, 5, 7, 9, 3]							
i = 5: chave = arr[5] = 3, j = 4							
5	4	arr[4]=9 > chave=3	sim	arr[5] = arr[4] (desloca)	[1, 2, 5, 7, 9, 9]	8	6
5	3	arr[3]=7 > chave=3	sim	arr[4] = arr[3] (desloca)	[1, 2, 5, 7, 7, 9]	9	7
5	2	arr[2]=5 > chave=3	sim	arr[3] = arr[2] (desloca)	[1, 2, 5, 5, 7, 9]	10	8
5	1	arr[1]=2 > chave=3	não	para, achou o lugar	[1, 2, 5, 5, 7, 9]	11	8
sai do laço (arr[1] <= chave); arr[2] = 3 -> [1, 2, 3, 5, 7, 9]							

Saída: [1, 2, 3, 5, 7, 9] **Comparações:** 11 **Trocas:** 8

Na coluna Trocas cada deslocamento conta como uma troca. O 9 (i = 2) entrou sem deslocar ninguém; o 1 (i = 3) andou até o começo do array, deslocando 3 elementos. Durante os deslocamentos aparece um valor repetido no array: é a casa da chave, que só é preenchida quando o laço termina.

4. Merge Sort

i	j	k	Compara	Escreve	Array depois	Comp.	Trocas
mergeSort(l=0, r=5): [5, 2, 9, 1, 7, 3], m = 2 -> divide em [0..2] e [3..5]							
mergeSort(l=0, r=2): [5, 2, 9], m = 1 -> divide em [0..1] e [2..2]							
mergeSort(l=0, r=1): [5, 2], m = 0 -> divide em [0..0] e [1..1]							
merge(l=0, m=0, r=1): L = [5] R = [2]							
0	0	0	L[0]=5 <= R[0]=2 (não)	arr[0] = 2	[2, 2, 9, 1, 7, 3]	1	1
0	-	1	R acabou, copia o resto de L	arr[1] = 5	[2, 5, 9, 1, 7, 3]	1	2
resultado: arr[0..1] = [2, 5]							
merge(l=0, m=1, r=2): L = [2, 5] R = [9]							
0	0	0	L[0]=2 <= R[0]=9 (sim)	arr[0] = 2	[2, 5, 9, 1, 7, 3]	2	3
1	0	1	L[1]=5 <= R[0]=9 (sim)	arr[1] = 5	[2, 5, 9, 1, 7, 3]	3	4
-	0	2	L acabou, copia o resto de R	arr[2] = 9	[2, 5, 9, 1, 7, 3]	3	5
resultado: arr[0..2] = [2, 5, 9]							
mergeSort(l=3, r=5): [1, 7, 3], m = 4 -> divide em [3..4] e [5..5]							
mergeSort(l=3, r=4): [1, 7], m = 3 -> divide em [3..3] e [4..4]							
merge(l=3, m=3, r=4): L = [1] R = [7]							
0	0	3	L[0]=1 <= R[0]=7 (sim)	arr[3] = 1	[2, 5, 9, 1, 7, 3]	4	6
-	0	4	L acabou, copia o resto de R	arr[4] = 7	[2, 5, 9, 1, 7, 3]	4	7
resultado: arr[3..4] = [1, 7]							
merge(l=3, m=4, r=5): L = [1, 7] R = [3]							
0	0	3	L[0]=1 <= R[0]=3 (sim)	arr[3] = 1	[2, 5, 9, 1, 7, 3]	5	8
1	0	4	L[1]=7 <= R[0]=3 (não)	arr[4] = 3	[2, 5, 9, 1, 3, 3]	6	9
1	-	5	R acabou, copia o resto de L	arr[5] = 7	[2, 5, 9, 1, 3, 7]	6	10
resultado: arr[3..5] = [1, 3, 7]							
merge(l=0, m=2, r=5): L = [2, 5, 9] R = [1, 3, 7]							
0	0	0	L[0]=2 <= R[0]=1 (não)	arr[0] = 1	[1, 5, 9, 1, 3, 7]	7	11
0	1	1	L[0]=2 <= R[1]=3 (sim)	arr[1] = 2	[1, 2, 9, 1, 3, 7]	8	12
1	1	2	L[1]=5 <= R[1]=3 (não)	arr[2] = 3	[1, 2, 3, 1, 3, 7]	9	13
1	2	3	L[1]=5 <= R[2]=7 (sim)	arr[3] = 5	[1, 2, 3, 5, 3, 7]	10	14
2	2	4	L[2]=9 <= R[2]=7 (não)	arr[4] = 7	[1, 2, 3, 5, 7, 7]	11	15
2	-	5	R acabou, copia o resto de L	arr[5] = 9	[1, 2, 3, 5, 7, 9]	11	16
resultado: arr[0..5] = [1, 2, 3, 5, 7, 9]							

Saída: [1, 2, 3, 5, 7, 9] **Comparações:** 11 **Trocas:** 16

Para 6 elementos acontecem 5 intercalações. As 16 trocas são escritas no array: 2 + 3 + 2 + 3 + 6. As 11 comparações vêm de 1 + 2 + 1 + 2 + 5.

5. Quick Sort

j	i	Compara	Resultado	Ação	Array depois	Comp.	Trocas
quickSort(lo=0, hi=5): pivô = arr[5] = 3, i = -1							
0	-1	arr[0]=5 <= pivô=3	não	-	[5, 2, 9, 1, 7, 3]	1	0
1	0	arr[1]=2 <= pivô=3	sim	i = 0; troca arr[0] e arr[1]	[2, 5, 9, 1, 7, 3]	2	1
2	0	arr[2]=9 <= pivô=3	não	-	[2, 5, 9, 1, 7, 3]	3	1
3	1	arr[3]=1 <= pivô=3	sim	i = 1; troca arr[1] e arr[3]	[2, 1, 9, 5, 7, 3]	4	2
4	1	arr[4]=7 <= pivô=3	não	-	[2, 1, 9, 5, 7, 3]	5	2
-	1	posiciona o pivô		troca arr[2] e arr[5] (pivô vai para a posição 2)	[2, 1, 3, 5, 7, 9]	5	3
p = 2 -> esquerda: quickSort(0, 1) direita: quickSort(3, 5)							
quickSort(lo=0, hi=1): pivô = arr[1] = 1, i = -1							
0	-1	arr[0]=2 <= pivô=1	não	-	[2, 1, 3, 5, 7, 9]	6	3
-	-1	posiciona o pivô		troca arr[0] e arr[1] (pivô vai para a posição 0)	[1, 2, 3, 5, 7, 9]	6	4
p = 0 -> esquerda: quickSort(0, -1) (retorna) direita: quickSort(1, 1) (retorna)							
quickSort(lo=3, hi=5): pivô = arr[5] = 9, i = 2							
3	3	arr[3]=5 <= pivô=9	sim	i = 3; i == j, sem troca	[1, 2, 3, 5, 7, 9]	7	4
4	4	arr[4]=7 <= pivô=9	sim	i = 4; i == j, sem troca	[1, 2, 3, 5, 7, 9]	8	4
-	4	posiciona o pivô		pivô já está na posição 5, sem troca	[1, 2, 3, 5, 7, 9]	8	4
p = 5 -> esquerda: quickSort(3, 4) direita: quickSort(6, 5) (retorna)							
quickSort(lo=3, hi=4): pivô = arr[4] = 7, i = 2							
3	3	arr[3]=5 <= pivô=7	sim	i = 3; i == j, sem troca	[1, 2, 3, 5, 7, 9]	9	4
-	3	posiciona o pivô		pivô já está na posição 4, sem troca	[1, 2, 3, 5, 7, 9]	9	4
p = 4 -> esquerda: quickSort(3, 3) (retorna) direita: quickSort(5, 4) (retorna)							

Saída: [1, 2, 3, 5, 7, 9] **Comparações:** 9 **Trocas:** 4

Na primeira partição o pivô 3 foi para a posição 2, deixando 2 elementos à esquerda e 3 à direita. O lado direito, [5, 7, 9], já estava em ordem, então só teve comparações e nenhuma troca.

Resumo (entrada [5, 2, 9, 1, 7, 3])

Algoritmo	Comparações	Trocas	Observação
Bubble Sort	14	8	parou na passada 4 por causa da flag
Selection Sort	15	2	sempre n(n-1)/2 comparações, poucas trocas
Insertion Sort	11	8	trocas = deslocamentos
Merge Sort	11	16	trocas = escritas no array
Quick Sort	9	4	pivô no último elemento